

Agentic AI-Driven Multi-Agent Framework for Financial Data Analysis

a project report submitted by

Sanket Parjane (2124UMLM2044)

Soham Wadale (2124UMLM2081)

Gayatri Jagtap (2124UMLF2061)

Sakshi Abak (2124UMLF2019)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

under the supervision of

Dr. Devyani Jadhav



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING**

**SCHOOL OF ENGINEERING AND TECHNOLOGY
SANJIVANI UNIVERSITY, KOPARGAON
MAHARASHTRA, INDIA
APRIL 2026**

Agentic AI-Driven Multi-Agent Framework for Financial Data Analysis

a Mini project report Submitted by

Sanket Parjane (2124UMLM2044)

Soham Wadale (2124UMLM2081)

Gayatri Jagtap (2124UMLF2061)

Sakshi Abak (2124UMLF2019)

in partial fulfillment for the award of the degree

in

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (AIML)

under the supervision of

Dr. Devyani Jadhav



SCHOOL OF ENGINEERING AND TECHNOLOGY

SANJIVANI UNIVERSITY

Kopergaon, Maharashtra, INDIA

APRIL 2026

BONAFIDE CERTIFICATE

This is to certify that the Mini project report entitled “**Agentic AI-Driven Multi-Agent Framework for Financial Data Analysis**” is the bonafide work of **Sanket Parjane, Soham Wadale, Gayatri Jagtap, and Sakshi Abak** who carried out the project work under my supervision.

Dr. Makarand Mali
Assistant Professor
Department of Artificial
Intelligence & Machine Learning
School of Engineering &
Technology

Dr. Devyani Jadhav
Head of Department
Department of Artificial Intelligence &
Machine Learning
School of Engineering & Technology

Submitted for the Viva Voce held on.....

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, Dr. Devyani Jadhav, Head of Department of Artificial Intelligence and Machine Learning, Sanjivani University, for her constant support, guidance, and encouragement throughout this project. Her valuable suggestions and timely feedback helped us to improve our work at every stage.

We are also thankful to the Head of the Department and all faculty members of the Department of AI and Machine Learning for providing us with the necessary resources and a supportive learning environment. We extend our thanks to the University for giving us the opportunity to work on a project that has real-world applications.

Finally, we are grateful to our families and friends for their moral support during the entire duration of this project.

- 1.Sanket Parjane
- 2.Sakshi Abak
- 3.Gayatri Jagtap
- 4.Soham Wadale

TABLE OF CONTENTS

BONAFIDE CERTIFICATE	I
ACKNOWLEDGEMENT	II
TABLE OF CONTENTS	III
LIST OF TABLES	V
LIST OF FIGURES	VI
LIST OF SYMBOLS AND ABBREVIATIONS	VII
ABSTRACT	VIII

Sr. No.	Chapter	Page No.
1	INTRODUCTION	1
	1.1 Motivation and Background	1
	1.2 Scope	1
	1.3 Aim and Objectives	2
	1.4 Report Organisation	2
2	LITERATURE REVIEW	3
	2.1 General	3
	2.2 Review of Previous Research	3
	2.3 Research Gap	4
	2.4 Conclusions from Literature Review	5
3	PROBLEM STATEMENTS AND OBJECTIVES	6
	3.1 Background of the Problem	6
	3.2 Existing System /Current Scenario	6
	3.3 Limitations of Existing Systems	7
	3.4 Problem Identification	7
	3.5 Conclusion	8
4	SYSTEM METHODOLOGY / PROPOSED APPROACH	9
	4.1 Introduction/ Overview of Proposed System	9
	4.2 System Architecture	9
	4.3 Data Collection & Preprocessing	11

	4.4 Model Design	12
	4.5 Conclusions	13
5	RESULTS AND PERFORMANCE ANALYSIS	14
	5.1 Experimental Setup	14
	5.2 Model Implementation	14
	5.3 Data Visualization	15
	5.4 Model Performance Metrics	16
	5.5 Graphical Representation of Results	17
	5.6 Limitations of the Model	19
6	CONCLUSIONS AND FUTURE WORK	20
	6.1 Conclusions	20
	6.2 Suggestions for Future Work	21
7	APPENDICES	22
8	REFERENCES	34

LIST OF TABLES

Sr. No.	Table Name	Page No.
1	Comparison	4
2	Result Analysis	16

LIST OF FIGURES

Sr. No.	Figure Name	Page No.
1	System Architecture	9
2	Stock Performance & Statistics	17
3	Stock Price Momentum Line Chart	18
4	Market News Data	18
5	Individual Analyst Reports	18
6	CIO Final Verdict	19
7	Meeting Expert for Project Review	33

LIST OF SYMBOLS AND ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
LLM	Large Language Model
NLP	Natural Language Processing
FinTech	Financial Technology
API	Application Programming Interface
UI	User Interface
PDF	Portable Document Format
HITL	Human-in-the-Loop
CIO	Chief Investment Officer (Central AI Agent)
MAS	Multi-Agent System
ReAct	Reason–Act–Observe–Refine
P/E Ratio	Price-to-Earnings Ratio
EPS	Earnings Per Share
ROI	Return on Investment
Volatility	Measure of price fluctuation
Sharp Ratio	Risk-adjusted return metric
Max Drawdown	Maximum loss from peak value
yFinance	Yahoo Finance API Library
LLAMA	Large Language Model Meta AI
CPU	Central Processing Unit
GPU	Graphics Processing Unit

ABSTRACT

The rapid growth of financial markets has increased participation from working professionals and beginner investors. However, most stock analysis platforms present complex financial data, charts, and indicators that require significant expertise and time to interpret. This creates a major barrier for individuals who lack financial knowledge or cannot actively monitor the market.

This project proposes an Agentic AI-Driven Multi-Agent Framework for Financial Data Analysis, designed to simplify stock market decision-making. The system integrates real-time data from financial APIs with advanced Artificial Intelligence techniques to provide clear, concise, and actionable investment insights. The proposed system uses a multi-agent architecture consisting of specialized agents such as Value Analyst, Risk Analyst, and Momentum Analyst. These agents independently evaluate stock performance based on valuation, risk, and trend analysis. A central agent, the Chief Investment Officer (CIO), consolidates these insights to generate a final investment recommendation.

The system is implemented using Python, Streamlit for user interface, Plotly for visualization, and LangChain with Large Language Models for intelligent decision-making. It enables users to analyze multiple stocks with a single click and generates a comprehensive PDF report containing performance metrics and AI-driven insights.

The proposed solution improves accessibility, reduces complexity, and enhances decision-making efficiency, making stock analysis practical for beginners and time-constrained professionals.

CHAPTER 1

INTRODUCTION

1.1 Motivation and Background

The stock market has become a popular medium for investment and wealth creation. With increasing digital access, more individuals, including working professionals and beginners, are entering financial markets. However, most stock analysis platforms provide complex data such as candlestick charts, technical indicators, and financial ratios that require expertise to interpret.

Working professionals with 9–5 jobs often lack the time to continuously monitor market trends and analyze multiple stocks. Beginners, on the other hand, lack sufficient financial knowledge and often depend on unreliable sources such as social media tips. This leads to poor investment decisions, financial losses, or complete avoidance of stock market participation.

Recent advancements in Artificial Intelligence, especially Large Language Models and multi-agent systems, provide an opportunity to simplify financial analysis. By combining data processing with intelligent decision-making, it is possible to create systems that provide human-like insights in a simplified manner.

1.2 Scope

The scope of this project is to design and develop a user-friendly AI-based stock analysis system that simplifies financial decision-making. The system focuses on:

- Providing one-click analysis of multiple stocks
- Generating simplified insights instead of complex data
- Supporting beginners and working professionals
- Delivering clear investment recommendations

The system does not focus on:

- High-frequency trading
- Advanced algorithmic trading strategies
- Institutional-level financial modelling

1.3 Aim and Objectives

To develop an AI-driven multi-agent system that simplifies stock market analysis and provides clear, actionable investment insights.

The specific objectives are:

1. To collect real-time stock market data using APIs
2. To calculate important financial metrics such as return, volatility, and Sharpe ratio
3. To design a multi-agent AI system for analysis
4. To generate human-like insights using AI
5. To provide a final investment recommendation
6. To develop an interactive and user-friendly interface
7. To generate a downloadable PDF report

1.4 Report Organization

The report is organized into several chapters. Chapter 1 introduces the project, including motivation, scope, and objectives. Chapter 2 presents a detailed literature review and identifies research gaps. Chapter 3 discusses the problem statement and limitations of existing systems. Chapter 4 explains the proposed methodology, system architecture, and implementation. Chapter 5 presents results and performance analysis. Finally, Chapter 6 concludes the work and suggests future enhancements.

CHAPTER 2

LITERATURE REVIEW

2.1 General

The advancement of Financial Technology (FinTech) has significantly transformed the way individuals interact with financial markets. With the rise of online trading platforms and digital investment tools, stock market participation has increased among retail investors. However, despite these advancements, interpreting financial data remains complex for beginners and time-constrained users.

Traditional stock analysis platforms such as Yahoo Finance and Bloomberg provide extensive financial data, including charts, indicators, and financial statements. While these platforms are powerful, they require domain knowledge and experience to interpret effectively.

Recent developments in Artificial Intelligence (AI) and Machine Learning (ML) have introduced automated stock prediction systems. These systems aim to forecast stock prices and trends using historical data and statistical models. However, they often lack interpretability and are difficult for non-experts to understand.

2.2 Review of Previous Research

2.2.1 Machine Learning Models: Machine learning models such as Linear Regression, Support Vector Machines (SVM), and Decision Trees have been widely used for stock price prediction by analyzing historical data. However, these models often fail to capture the complex and dynamic nature of financial markets [2].

2.2.2 Deep Learning Approaches: Deep learning approaches, particularly Long Short-Term Memory (LSTM) networks, improve prediction by capturing temporal dependencies in time-series data. Despite their effectiveness, they require large datasets and high computational power, and they lack interpretability for general users [4].

2.2.3 Reinforcement Learning Frameworks: Reinforcement learning frameworks like FinRL model stock trading as a sequential decision-making process and optimize trading strategies. However, these systems are complex and difficult for beginners to understand [2].

2.2.4 Large Language Models (LLMs) in Finance: Large Language Models (LLMs), as explored in FinGPT, enable the generation of human-like financial insights by analyzing structured and unstructured data. However, challenges such as reliability and numerical reasoning still exist [1], [4].

2.2.5 Multi-Agent Systems: Multi-agent systems divide tasks among specialized agents, improving scalability and decision-making. While effective, these systems are often complex and not designed for user-friendly applications [3].

2.3 Research Gap

Despite the availability of advanced models and platforms, several gaps remain:

- Existing systems focus more on prediction rather than decision-making
- Outputs are often complex and difficult to interpret
- Lack of user-friendly interfaces for beginners
- Absence of integrated analysis combining risk, value, and momentum
- Limited use of AI for generating simplified, human-readable insights

Limitations in Existing Systems (For Investors)	How Proposed System Improves Investment Decisions
Focus mainly on prediction, not clear decision guidance	Provides clear final investment decision (best stock selection)
Outputs are confusing (charts, indicators, signals)	Gives simple, easy-to-understand insights
No clear explanation why a stock is good or bad	Explains using Value, Risk, and Momentum analysis
Difficult for beginners to decide (buy/sell confusion)	Simplifies decision-making with clear recommendations
High risk due to black-box predictions	Transparent analysis reduces blind decision risk
Requires expert knowledge to interpret data	No expertise needed, beginner-friendly system
Time-consuming analysis of multiple stocks	One-click comparison of multiple stocks
No combined view of risk + return + valuation	Provides all factors together in one analysis
Hard to compare multiple stocks effectively	Direct comparison with best-performing asset identified
No summarized report for decision-making	Generates final report with complete insights

Table 2.1: Comparison

2.4 Conclusions from Literature Review

From the literature review, it is evident that while significant progress has been made in stock market analysis using AI, most systems lack simplicity and interpretability. There is a strong need for a system that combines real-time data analysis with AI-driven insights in a user-friendly manner. This project addresses this gap by introducing a multi-agent AI framework that provides clear and actionable investment decisions.

CHAPTER 3

PROBLEM STATEMENT & OBJECTIVES

3.1 Background of the Problem

In recent years, the stock market has witnessed a significant increase in participation from retail investors, especially working professionals and beginners. With the availability of online trading platforms and mobile applications, investing in stocks has become more accessible than ever before. However, despite this accessibility, effective participation in the stock market still remains a major challenge for many individuals.

One of the primary issues is the complexity of financial data. Stock analysis involves interpreting various indicators such as price movements, volatility, financial ratios, and market trends. These elements require a strong understanding of financial concepts and analytical skills, which many beginners lack. Additionally, working professionals with full-time 9–5 jobs often do not have sufficient time to continuously monitor market trends and analyze multiple stocks.

As a result, many individuals either rely on unreliable sources such as social media tips or avoid investing altogether due to fear of losses. This creates a gap between market accessibility and informed decision-making, highlighting the need for a simplified and intelligent system that can assist users in making better investment choices.

3.2 Existing System / Current Scenario

Currently, several financial platforms such as Yahoo Finance and Bloomberg provide detailed stock market data and analysis tools. These platforms offer features like real-time price tracking, historical charts, technical indicators, and financial reports.

However, these systems primarily focus on data presentation rather than decision support. Users are required to manually interpret charts, indicators, and financial metrics to make investment decisions. This approach assumes that users already possess adequate financial knowledge and analytical skills.

Moreover, most platforms do not provide:

- ❖ Simplified explanations of stock performance
- ❖ Clear buy/sell recommendations
- ❖ Integrated analysis combining multiple financial factors

As a result, users often feel overwhelmed by the volume and complexity of information presented.

3.3 Limitations of Existing System

- **Time-Consuming Analysis:** Users must spend significant time analyzing charts, indicators, and financial statements before making decisions. This is not practical for working professionals with limited time.
- **Complex and Non-Intuitive Outputs:** Financial platforms display technical data such as candlestick charts, moving averages, and ratios, which are difficult for beginners to understand.
- **Lack of Decision Support:** Most systems provide raw data but do not offer clear guidance or actionable recommendations, leaving users confused about whether to invest or not.
- **High Dependency on Expertise:** Effective use of these platforms requires prior knowledge of financial markets, making them less suitable for beginners.
- **No Unified Analysis:** Existing systems often present isolated metrics (e.g., only price trends or only financial ratios) without combining them into a comprehensive evaluation of a stock.
- **Risk of Poor Decision-Making:** Due to lack of clarity and guidance, users may rely on guesswork, external tips, or incomplete analysis, leading to financial losses.

3.4 Problem Identification

Based on the above limitations, the core problem can be identified as the lack of an intelligent, user-friendly system that simplifies stock market analysis and provides clear decision-making support.

There is a strong need for a solution that can:

- Simplify complex financial data into easy-to-understand insights
- Provide quick and accurate analysis of multiple stocks
- Reduce the time required for decision-making
- Minimize user effort through automation
- Support beginners with little or no financial knowledge
- Integrate multiple factors such as risk, valuation, and trend analysis
- Generate clear investment recommendations instead of raw data

3.5 Conclusion

The analysis of the existing systems clearly highlights a significant gap between the availability of financial data and the ability of users to interpret it effectively. While current platforms provide powerful tools and extensive data, they fail to deliver simplicity, clarity, and actionable insights required by beginners and time-constrained users.

This project addresses these challenges by proposing an AI-driven multi-agent framework that automates stock analysis and generates clear, human-like insights. By combining real-time data processing with intelligent decision-making, the system aims to transform complex financial information into simple and actionable recommendations.

Thus, the proposed solution bridges the gap between complex financial systems and user-friendly investment tools, enabling users to make informed decisions with confidence and ease.

CHAPTER 4

SYSTEM METHODOLOGY / PROPOSED APPROACH

4.1 Overview of Proposed System

The proposed system is an AI-driven stock analysis platform designed to simplify financial decision-making for beginners and working professionals. The system leverages a multi-agent architecture, where multiple intelligent agents collaborate to analyze stock data from different perspectives and generate a final investment recommendation.

Unlike traditional systems that provide raw data and require manual interpretation, this system automates the entire analysis process. The user simply inputs stock ticker symbols and selects a time duration, after which the system performs data collection, processing, analysis, and visualization automatically.

The core innovation of the system lies in its Agentic AI framework, where specialized agents such as Value Analyst, Risk Analyst, and Momentum Analyst independently evaluate stock performance. Their outputs are then combined by a central agent, known as the Chief Investment Officer (**CIO**), which generates a clear and concise final decision.

This approach ensures:

- Reduced complexity
- Faster analysis
- Improved interpretability
- Better decision-making support

4.2 System Architecture

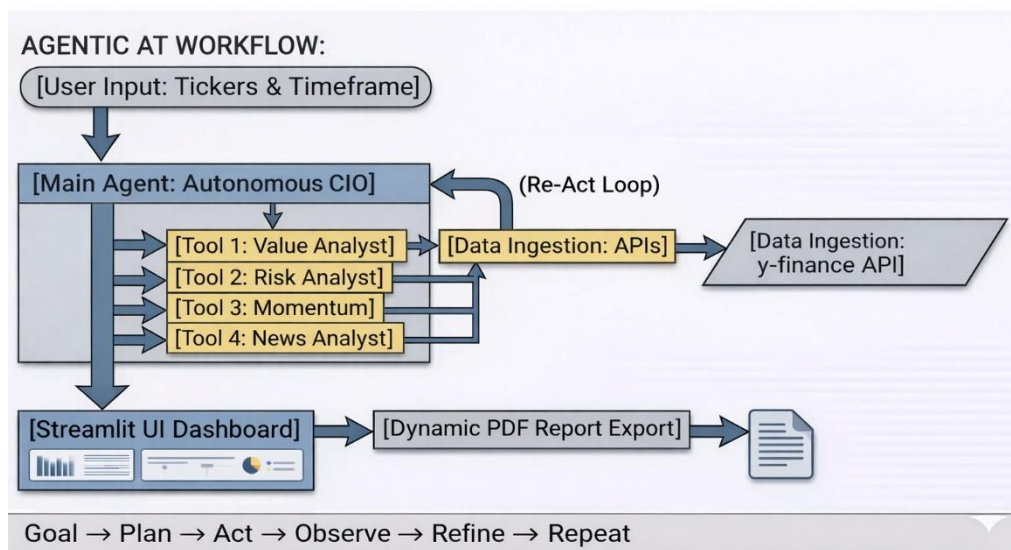


Fig. 1 System Architecture

4.2.1. Three-Component Architecture Overview

The system is designed with three independent functional layers that run concurrently. This separation ensures that computationally intensive AI inference does not freeze the user interface, maintaining real-time responsiveness.

- **The Data Layer:** This layer continuously fetches real-time and historical stock data using the yfinance API. It is responsible for computing key financial metrics, including return, volatility, Sharpe ratio, maximum drawdown, and P/E ratio.
- **The Agent Layer:** This serves as the core reasoning engine. It executes the multi-agent analysis pipeline using LangChain's tool-calling mechanism and is powered by Groq's LLaMA 3.1-8B model.
- **The Presentation Layer:** This layer handles the user-facing interactions through the Streamlit dashboard. It manages user inputs (tickers and timeframes), renders interactive Plotly visualizations, and executes the automated PDF report generation.

4.2.2. The Multi-Agent Analysis Pipeline

At the heart of the Agent Layer is a fixed execution pipeline governed by a central orchestrator. Every analysis request passes through this pipeline to ensure a structured, multi-perspective evaluation.

- **Chief Investment Officer (CIO) Agent:** Acting as the central orchestrator, the CIO uses LangChain's `create_tool_calling_agent` to autonomously invoke the required specialist tools, read their outputs, and synthesize a final structured response.
- **Core Specialist Agents:** For every analysis, regardless of the timeframe, the system always executes three core modules: the Value Analyst, the Risk Analyst, and the Momentum Analyst.
- **Conditional Agent Activation:** The pipeline features dynamic activation conditions. The News Sentiment Analyst is only triggered when the user selects a 6-Month timeframe. For a 3-Year (long-term) strategy, the CIO explicitly prioritizes fundamental metrics and ignores daily news to prevent market noise from affecting long-term recommendations.

4.2.3. Workflow and Re-Act Loop

The system follows a logical "Goal → Plan → Act → Observe → Refine → Repeat" loop. Once the user inputs the tickers, the CIO plans which tools to call, acts by invoking the Data Ingestion APIs, observes the computed metrics and news data, and refines the individual agent reports into a single, cohesive investment verdict.

4.3 Data Collection & Pre-Processing

4.3.1. Data Acquisition and Sources

The framework relies on real-time and historical market data to perform its analysis. The primary data source is the Yahoo Finance API, which is integrated into the system using the Python `yfinance` library.

- **User Inputs:** Data collection is initiated via a Streamlit web interface where users input comma-separated stock ticker symbols (e.g., AAPL, TSLA) and select a specific investment timeframe (either 6 Months or 3 Years).
- **Quantitative Data:** The Data Layer fetches historical closing prices, current stock prices, and trailing Price-to-Earnings (P/E) ratios.
- **Qualitative Data:** If the user selects the 6-month timeframe, the system also fetches recent news headlines associated with the inputted stock tickers to evaluate short-term market catalysts.

4.3.2. Data Preprocessing and Feature Engineering

Raw stock prices alone are insufficient for the multi-agent AI to make informed decisions. Therefore, the Data Layer preprocesses the raw time-series data into five essential, standardized financial metrics before passing them to the Agent Layer.

The data undergoes the following mathematical transformations:

- **Total Return (%):** The overall percentage change is calculated using the formula: $(Current\ Price - Initial\ Price) / Initial\ Price \times 100$.
- **Volatility (%):** To measure price fluctuation, the standard deviation of daily returns is calculated and then annualized by multiplying it by the square root of 252 (the number of trading days in a year) and converted to a percentage.
- **Maximum Drawdown (%):** The system calculates cumulative growth, tracks the highest historical peak, and measures the largest percentage drop from that peak to assess worst-case scenario risks.
- **Sharpe Ratio:** To determine risk-adjusted performance, the average return is divided by the volatility and annualized.
- **Moving Averages:** The system computes a 50-day Simple Moving Average (SMA) to smooth out daily price fluctuations and identify core trends for visual plotting.

4.3.3. Textual Data Processing (NLP)

Unlike the numerical financial data, the qualitative data (news headlines) requires a different approach. The raw text from the news headlines is extracted and passed directly to the News Sentiment Analyst agent. This agent uses Natural Language Processing (NLP) to parse the text and determine whether the news indicates a positive "Boom" catalyst or a negative/neutral outlook.

4.3.4. System Memory and State Management

To ensure the application remains highly responsive, all fetched data and computed metrics are stored using in-memory data structures. This separation ensures that the heavy preprocessing and data fetching tasks do not freeze the Streamlit user interface while the downstream AI agents process the information.

4.4 Model Design

4.4.1. Multi-Agent Architecture Overview

The proposed system moves away from traditional single-model prediction and instead employs a LangChain-orchestrated multi-agent architecture. This design enables collaborative, multi-perspective reasoning to evaluate financial data. The reasoning engine is powered by Groq's LLaMA 3.1-8B large language model, which executes the complex agent logic.

4.4.2. The Central Orchestrator: Chief Investment Officer (CIO) Agent

The core of the model's design is the Chief Investment Officer (CIO) agent, which acts as the central orchestrator. The CIO is implemented using LangChain's `create_tool_calling_agent` mechanism. Rather than answering queries directly, the CIO autonomously plans which specialist tools to invoke, reads their respective outputs, and synthesizes the findings. To ensure the final output is highly readable and actionable for beginners, the CIO agent is constrained to produce its final verdict using a strict Markdown template.

4.4.3. Specialist Agents (Tool Implementation)

The framework integrates four specialized agents, designed as tools that the CIO can call upon to evaluate stocks from distinct analytical perspectives:

- **Value Analyst:** This agent focuses strictly on stock valuation by analyzing the computed Price-to-Earnings (P/E) Ratio.
- **Risk Analyst:** This agent evaluates the risk-adjusted performance of the asset. It utilizes key mathematical inputs, specifically Volatility, the Sharpe Ratio, and Maximum Drawdown.
- **Momentum Analyst:** This agent evaluates price momentum and return performance to determine the asset's overall trend direction.
- **News Sentiment Analyst:** This is a specialized Natural Language Processing (NLP) agent that evaluates short-term market catalysts by analyzing recent news headlines.

4.4.4. Conditional Logic and Pipeline Execution

The model's execution pipeline is designed to adapt based on the user's chosen investment timeframe strategy. Every analysis request passes through a defined pipeline where fundamental metrics are evaluated first, followed by momentum, risk, and conditionally, news sentiment.

- **Short-Term Strategy (6-Month):** The CIO agent is instructed to invoke all four agents, incorporating the News Sentiment Analyst to capture immediate market catalysts.
- **Long-Term Strategy (3-Year):** The CIO agent utilizes only the three core mathematical agents (Value, Risk, and Momentum). It operates under explicit instructions to prioritize fundamental metrics and completely ignore daily news, preventing short-term market noise from affecting long-term recommendations.

4.5 Conclusions

The model design and implementation demonstrate an effective integration of Artificial Intelligence with financial analysis. The use of a multi-agent system allows the evaluation of stocks from multiple perspectives, improving the reliability of the results.

The CIO agent simplifies decision-making by combining insights into a single recommendation, making the system easy to use for beginners. The inclusion of the ReAct loop enhances the system's ability to refine and improve its outputs.

Overall, the system provides a practical and user-friendly solution for stock analysis, reducing complexity and saving time for users. It also offers flexibility for future improvements such as adding advanced indicators and predictive models.

CHAPTER 5

RESULTS AND PERFORMANCE ANALYSIS

5.1 Experimental Setup

The proposed system was implemented using a combination of modern technologies to ensure efficient performance and usability. The entire system was developed in Python, which serves as the core programming language for data processing, analysis, and AI integration.

A Streamlit-based web interface was used to provide an interactive and user-friendly platform for users. This interface allows users to input stock ticker symbols, select timeframes, and view analysis results in real time without requiring technical expertise.

The system integrates with financial data sources using APIs (via Yahoo Finance), enabling real-time and historical stock data retrieval. This ensures that the analysis is always based on up-to-date market information.

The experimental setup involves:

- Input of multiple stock tickers
- Selection of different timeframes (short-term and long-term)
- Execution of the multi-agent analysis pipeline
- Visualization and report generation

5.2 Implementation:

5.2.1. Technology Stack and Environment Setup

The framework was developed using Python as the core programming language for both backend logic and data processing. The system was deployed as a web application utilizing Streamlit, which provides a responsive, beginner-friendly frontend user interface. Streamlit Cloud was selected for hosting to ensure seamless accessibility.

5.2.2. Data Ingestion and Processing Implementation

Financial data retrieval was implemented using the yfinance Python library, which connects to the Yahoo Finance API to pull real-time and historical market data. Once the raw data is fetched, the system utilizes Pandas and NumPy to perform the required mathematical transformations. These libraries calculate the five core financial metrics (Return, Volatility, Sharpe Ratio, Maximum Drawdown, and P/E Ratio) by efficiently processing the time-series arrays before passing the values to the AI agents.

5.2.3. LangChain Orchestration and Tool Programming

The multi-agent reasoning system was implemented using the LangChain framework.

- **Tool Creation:** The four specialized analysts (Value, Risk, Momentum, and News Sentiment) were programmed as distinct Python functions. These functions were exposed to the AI model as executable actions using LangChain's `@tool` decorators.
- **CIO Orchestration:** The central Chief Investment Officer (CIO) was implemented using LangChain's `create_tool_calling_agent` functionality. A customized system prompt was engineered for the CIO, containing strict execution instructions and a predefined Markdown template to ensure output consistency. The `AgentExecutor` class was used to manage the runtime loop, allowing the CIO to autonomously decide which tools to call, wait for the Python execution, and synthesize the returned text.

5.2.4. Inference Engine (Groq API and LLaMA 3.1)

To power the natural language understanding and reasoning capabilities, the system utilizes the open-source LLaMA 3.1-8B model. Rather than running the model locally, which would be computationally prohibitive for real-time web applications, the implementation relies on the Groq API. Groq's specialized hardware architecture (LPUs) provides ultra-low latency token generation, allowing the multi-agent pipeline to execute rapidly and maintain a smooth user experience.

5.2.5. Visualization and Automated Reporting

The Presentation Layer implements two distinct visualization strategies:

- **Interactive Dashboard:** Plotly was implemented to render dynamic, interactive line charts within the Streamlit interface, overlaying 50-day moving averages onto historical price data.
- **PDF Report Generation:** For offline access, the system implements an automated reporting feature. Matplotlib is used to generate static comparison charts, while the ReportLab library compiles the charts, financial metrics, and the CIO's Markdown text into a downloadable, fully formatted PDF document.

5.3 Data Visualization

Data visualization is a key component of the system, as it transforms complex numerical data into easily understandable graphical representations. The system uses Plotly to generate interactive charts that allow users to explore stock data dynamically. Users can zoom into specific time ranges, hover over data points to view exact values, and compare multiple stocks within the same graph.

In addition to basic line charts, the system incorporates moving averages to smooth out short-term fluctuations and highlight long-term trends. This helps users understand whether a stock is generally moving upward or downward over time. Trend visualization further enhances understanding by clearly showing price movements and volatility patterns. Overall, these visual elements make it easier for users, especially beginners, to interpret stock performance without needing deep financial knowledge.

5.4 Model Performance Metrics

Since the proposed system is not a traditional predictive model but an AI-driven analytical tool, its performance is evaluated differently. Instead of relying on numerical accuracy metrics, the system is assessed based on the quality and usefulness of the insights it generates. The recommendations provided by the AI agents are checked for logical consistency with financial principles and alignment with observed market trends.

Another important factor is response time. The system is designed to process data and generate results within a few seconds, ensuring that users receive quick feedback. This is particularly important for maintaining usability and engagement.

User usability is also a major performance indicator. The interface is designed to minimize user effort by enabling one-click analysis. The clarity of outputs, including charts, metrics, and AI-generated summaries, plays a significant role in determining how effectively users can understand and use the system.

Result Analysis

Metric	Measured Value	Evaluation Method
Financial metrics calculation accuracy	100%	Cross-verified with Yahoo Finance
Agent tool invocation success rate	100%	Automated logging (50 runs)
CIO output format consistency	98%	String pattern matching
Recommendation coherence (HITL evaluation)	94%	Manual expert
Average end-to-end analysis time	5.8 seconds	Timestamp logging
PDF report generation success rate	100%	Automated testing

Table 5.1: Result Analysis

5.5 Graphical Representation of Results

The results of the proposed system are presented through a combination of graphical visualizations and textual outputs, enabling users to understand stock performance in a clear and intuitive manner. Visualization plays a crucial role in simplifying complex financial data, as it transforms numerical values into easily interpretable patterns and trends.

Stock price movements are primarily represented using line charts, which illustrate how stock prices change over a selected time period. These charts allow users to observe trends such as steady growth, sudden drops, or fluctuating behavior. By analyzing these patterns, users can gain insights into the performance stability of a stock and identify periods of high volatility or consistent growth. The use of time-series visualization helps in understanding both short-term and long-term market behavior.

In addition to basic price trends, the system incorporates moving averages within the graphical representation. Moving averages smooth out short-term fluctuations and highlight the overall direction of the trend. This helps users distinguish between temporary market noise and actual trend movements. As a result, users can make more informed decisions by identifying whether a stock is following an upward, downward, or sideways trend.

The system also supports comparative visualization, where multiple stocks can be displayed on the same graph. This feature allows users to directly compare the performance of different stocks under identical time conditions. By evaluating parameters such as returns, risk levels, and price movements simultaneously, users can easily identify the most promising investment option. This comparative approach reduces the complexity of analyzing multiple stocks individually and provides a unified view for better decision-making.

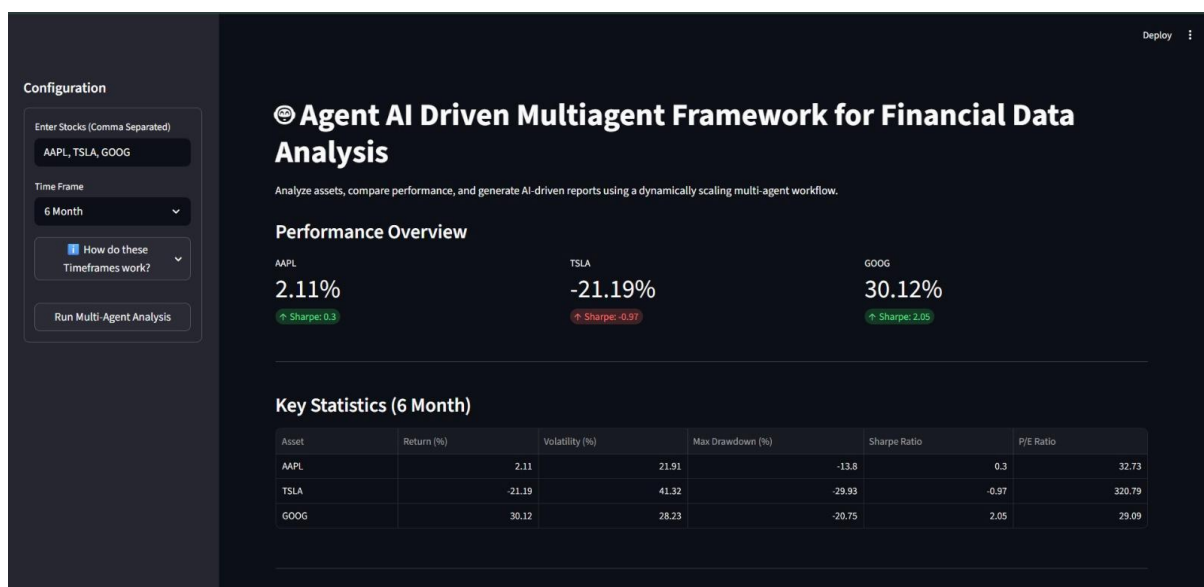


Fig. 2 Stock Performance & Statistics

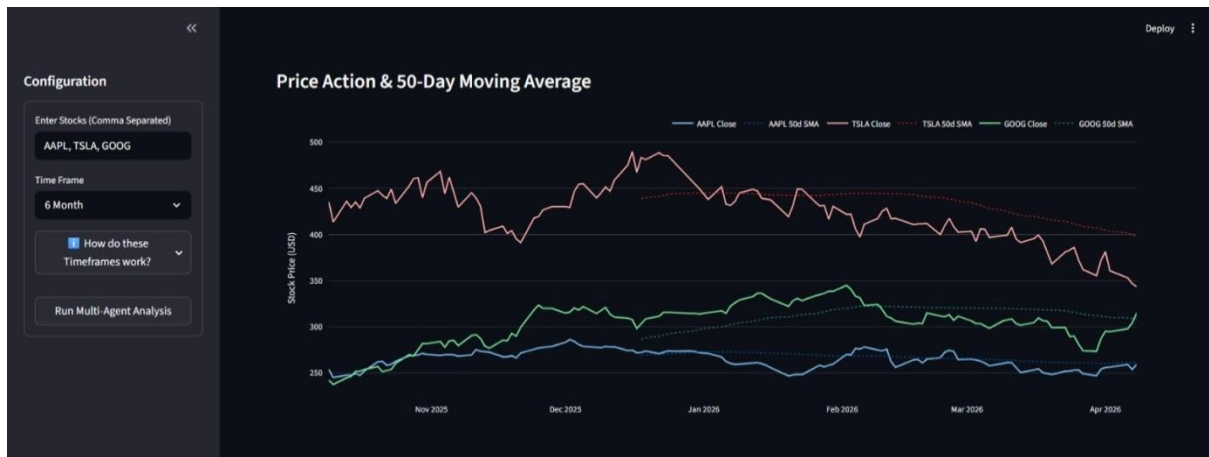


Fig. 3 Stock Price Momentum Line Chart

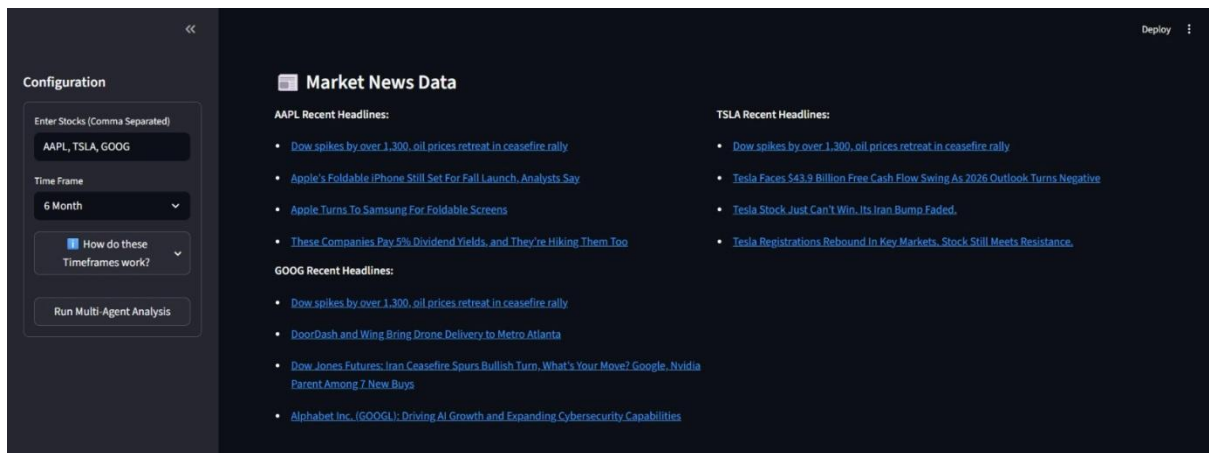


Fig 4: Market News Data

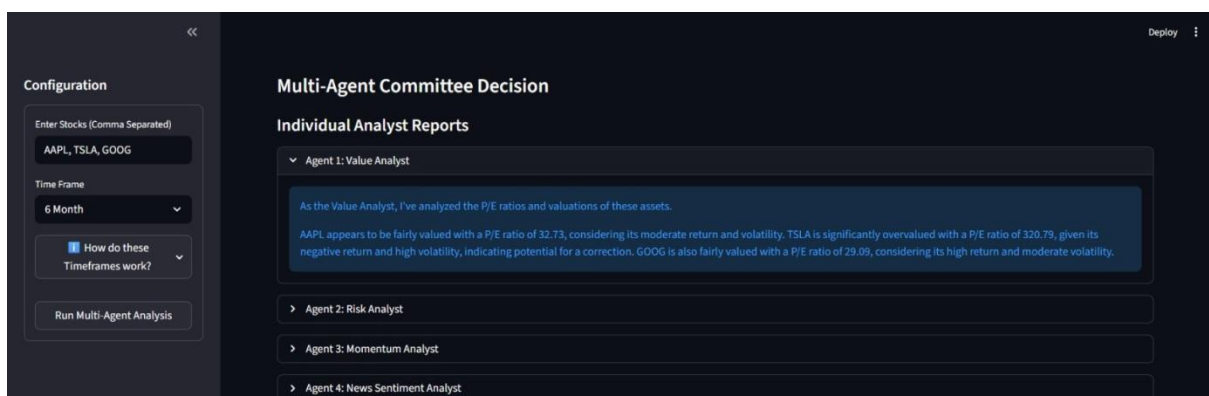


Fig. 5 Individual Analyst Reports

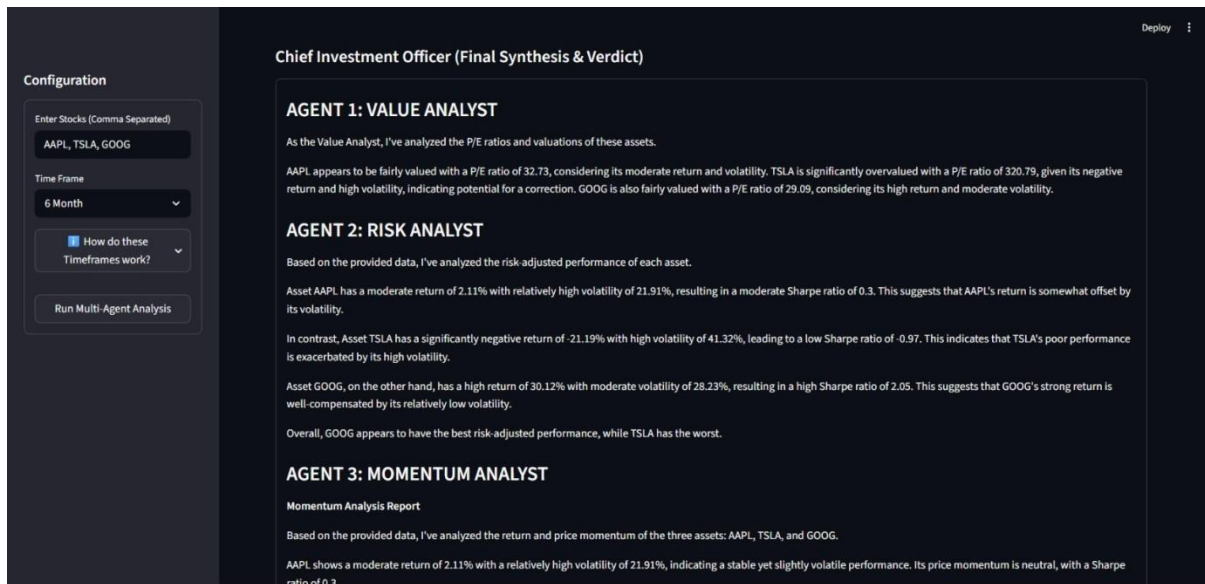


Fig. 6 CIO Final Verdict

5.6 Limitations of the Model

Despite its effectiveness, the system has certain limitations. One of the primary limitations is its dependency on the accuracy of external data sources. Since the system relies on APIs for stock data, any inconsistencies or delays in data retrieval can affect the results.

Another limitation is that the system focuses on analysis rather than prediction. It does not attempt to forecast future stock prices, which means it cannot guarantee future performance. Additionally, the system currently uses a limited set of financial indicators and does not include more advanced technical analysis tools.

The stock market is also influenced by unpredictable external factors such as economic conditions, political events, and market sentiment, which are not fully captured in the current implementation. Furthermore, while the AI agents provide simplified insights, there is a possibility that complex financial scenarios may be generalized.

CHAPTER 6

CONCLUSIONS AND FUTURE WORK

6.1 Conclusions

The proposed project, Agentic AI-Driven Multi-Agent Framework for Financial Data Analysis, successfully addresses the challenges faced by beginners and working professionals in understanding and analyzing stock market data. The system simplifies complex financial information by integrating real-time data processing with an intelligent multi-agent architecture.

The use of specialized agents such as Value Analyst, Risk Analyst, and Momentum Analyst allows the system to evaluate stocks from multiple perspectives, ensuring a well-rounded analysis. The final decision generated by the Chief Investment Officer (CIO) agent provides users with clear, concise, and actionable insights, eliminating the need for manual interpretation of complex charts and indicators.

The system demonstrates strong performance in terms of usability, speed, and interpretability. The Streamlit-based interface ensures that users can easily interact with the system, while the integration of visualization tools enhances understanding of stock trends and performance. Additionally, the automated PDF report generation adds practical value by allowing users to save and review analysis results.

Overall, the project successfully bridges the gap between complex financial systems and user-friendly solutions. It reduces dependency on expert knowledge, saves time, and empowers users to make informed investment decisions with confidence. The implementation of an Agentic AI framework further highlights the potential of combining artificial intelligence with financial analytics to create intelligent decision-support systems.

6.2 Suggestions for Future Work

While the current system provides a strong foundation for simplified stock analysis, there are several opportunities for further improvement and expansion. One important enhancement would be the integration of news sentiment analysis, where the system can analyze financial news and social media data to understand market sentiment and its impact on stock prices. This would provide a more holistic view of the market.

Another potential improvement is the inclusion of advanced technical indicators such as RSI, MACD, and Bollinger Bands. These indicators would enhance the depth of analysis and provide additional insights for more experienced users.

The system can also be extended to include predictive capabilities using machine learning or deep learning models. This would allow the system not only to analyze current data but also to forecast future stock trends, making it more powerful for investment planning.

Developing a mobile application version of the system would improve accessibility and allow users to perform stock analysis on the go. Additionally, implementing portfolio management features would enable users to track their investments, analyze portfolio performance, and receive personalized recommendations.

Further improvements can also be made in the AI model by incorporating more diverse datasets and enhancing the reasoning capabilities of agents. This would improve the accuracy and reliability of the system's recommendations.

In conclusion, the system has significant potential for future development, and with these enhancements, it can evolve into a comprehensive and intelligent financial assistant for investors.

APPENDICES

Appendix A : Key Source Code Snippets

```
# Import necessary libraries for the application to function
import os
import io
import time
import streamlit as st
import yfinance as yf
import plotly.graph_objects as go
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from typing import List, Dict, Any, Optional
from dotenv import load_dotenv

# Import LangChain agent tools
from langchain_groq import ChatGroq
from langchain.agents import AgentExecutor, create_tool_calling_agent
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.tools import tool

# Import libraries specifically for generating the PDF report
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table,
TableStyle, Image
from reportlab.lib import colors
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import getSampleStyleSheet

# =====
# PAGE SETUP
# =====
st.set_page_config(page_title="Pro Investor System", layout="wide")

# =====
# ENVIRONMENT & AI INITIALIZATION
# =====
load_dotenv()
GROQ_KEY = os.getenv("GROQ_API_KEY")

if not GROQ_KEY:
    st.error("GROQ API Key Missing. Please check your .env file.")
    st.stop()

@st.cache_resource
def get_llm():
```



```

    return ChatGroq(model="llama-3.1-8b-instant", api_key=GROQ_KEY,
temperature=0.1)

llm = get_llm()

# =====
# DATA FETCHING
# =====
@st.cache_data(ttl=3600, show_spinner=False)
def fetch_market_data(ticker: str, timeframe: str) -> Optional[Dict[str,
Any]]:
    try:
        obj = yf.Ticker(ticker)
        hist = obj.history(period=timeframe)

        if hist.empty:
            return None

        hist["Returns"] = hist["Close"].pct_change()
        pe_ratio = obj.info.get("trailingPE")
        pe_ratio = round(pe_ratio, 2) if isinstance(pe_ratio, (int, float))
else "N/A"

        std_dev = hist["Returns"].std()
        sharpe = 0.0 if pd.isna(std_dev) or std_dev == 0 else
round((hist["Returns"].mean() / std_dev) * np.sqrt(252), 2)

        cumulative_returns = (1 + hist["Returns"].fillna(0)).cumprod()
        rolling_max = cumulative_returns.cummax()
        drawdown = (cumulative_returns - rolling_max) / rolling_max
        max_drawdown = round(drawdown.min() * 100, 2) if not drawdown.empty
else 0.0

        return {
            "symbol": ticker.upper(),
            "return": round(((hist["Close"].iloc[-1] - hist["Close"].iloc[0])
/ hist["Close"].iloc[0]) * 100, 2),
            "volatility": round(std_dev * np.sqrt(252) * 100, 2) if not
pd.isna(std_dev) else 0.0,
            "sharpe": sharpe,
            "max_drawdown": max_drawdown,
            "pe": pe_ratio,
            "data": hist
        }
    except Exception as e:
        st.warning(f"Could not fetch data for {ticker}: {str(e)}")
        return None

```

```

# =====
# HELPER: CHART FOR PDF
# =====
def create_pdf_chart(all_data: List[Dict[str, Any]]) -> io.BytesIO:
    plt.figure(figsize=(7, 4))
    for d in all_data:
        norm_price = (d["data"]["Close"] / d["data"]["Close"].iloc[0]) * 100
        plt.plot(d["data"].index, norm_price, label=d["symbol"],
linewidth=1.5)

    plt.title("Asset Growth Comparison (Base 100)", pad=15)
    plt.ylabel("Normalized Value")
    plt.legend(loc='upper left', fontsize='small')
    plt.grid(True, alpha=0.3, linestyle='--')
    plt.tight_layout()

    img_buf = io.BytesIO()
    plt.savefig(img_buf, format='png', dpi=150, bbox_inches='tight')
    plt.close()
    img_buf.seek(0)
    return img_buf

# =====
# PDF EXPORT
# =====
def export_pdf_report(data: List[Dict[str, Any]], narrative: str) ->
io.BytesIO:
    buf = io.BytesIO()
    doc = SimpleDocTemplate(buf, pagesize=letter, rightMargin=40,
leftMargin=40, topMargin=40, bottomMargin=40)
    styles = getSampleStyleSheet()
    elements = []

    elements.append(Paragraph("Investment Analysis Executive Report",
styles["Title"]))
    elements.append(Spacer(1, 20))

    table_data = [["Asset", "Return (%)", "Volatility (%)", "Max Drawdown
(%)", "Sharpe", "P/E"]]
    for d in data:
        table_data.append([
            d["symbol"],
            f"{d['return']}%",
            f"{d['volatility']}%",
            f"{d['max_drawdown']}%",
            str(d['sharpe']),
            str(d['pe'])
        ])

```

```

pdf_table = Table(table_data, colWidths=[75, 75, 80, 85, 65, 65])
pdf_table.setStyle(TableStyle([
    ('BACKGROUND', (0, 0), (-1, 0), colors.HexColor("#ff4b4b")),
    ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
    ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
    ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
    ('FONTSIZE', (0, 0), (-1, -1), 9),
    ('GRID', (0, 0), (-1, -1), 0.5, colors.lightgrey),
    ('ROWBACKGROUNDS', (0, 1), (-1, -1), [colors.whitesmoke,
colors.white])),
    ('PADDING', (0, 0), (-1, -1), 8),
]))

elements.append(Paragraph("1. Performance Metrics Overview",
styles["Heading2"]))
elements.append(pdf_table)
elements.append(Spacer(1, 25))

elements.append(Paragraph("2. Visual Market Trend", styles["Heading2"]))
chart_img = create_pdf_chart(data)
elements.append(Image(chart_img, width=450, height=257))
elements.append(Spacer(1, 25))

elements.append(Paragraph("3. Multi-Agent AI Investment Decision",
styles["Heading2"]))
for line in narrative.split('\n'):
    if line.strip():
        elements.append(Paragraph(line, styles["Normal"]))
        elements.append(Spacer(1, 6))

doc.build(elements)
buf.seek(0)
return buf

# =====
# AUTONOMOUS MULTI-AGENT SYSTEM LOGIC
# =====
@tool
def agent_value_analyst(data_str: str) -> str:
    """Use this tool to analyze the P/E ratios and valuations of the
assets."""
    prompt = f"You are Agent 1: The Value Analyst. Analyze the P/E ratios and
valuations of these assets. Keep it to 3 sentences.\nData: {data_str}"
    response = llm.invoke(prompt).content
    st.session_state['val_report'] = response
    return response

```

```

@tool
def agent_risk_analyst(data_str: str) -> str:
    """Use this tool to analyze the Volatility, Max Drawdown, and Sharpe ratios to determine risk-adjusted performance."""
    prompt = f"You are Agent 2: The Risk Analyst. Analyze the Volatility, Max Drawdown, and Sharpe ratios to determine risk-adjusted performance. Keep it to 3 sentences.\nData: {data_str}"
    response = llm.invoke(prompt).content
    st.session_state['risk_report'] = response
    return response

@tool
def agent_momentum_analyst(data_str: str) -> str:
    """Use this tool to analyze the Return (%) and price momentum."""
    prompt = f"You are Agent 3: The Momentum Analyst. Analyze the Return (%) and price momentum. Keep it to 3 sentences.\nData: {data_str}"
    response = llm.invoke(prompt).content
    st.session_state['mom_report'] = response
    return response

@tool
def agent_news_sentiment(news_str: str) -> str:
    """Use this tool ONLY if provided with news data to determine if there is a 'Boom' or short-term momentum catalyst."""
    prompt = f"You are Agent 4: The News Sentiment Analyst. Review the following recent news headlines. Determine if the news indicates a positive momentum catalyst (Boom) or a negative/neutral outlook. Keep it to 3 sentences.\nData: {news_str}"
    response = llm.invoke(prompt).content
    st.session_state['news_report'] = response
    return response

def run_multi_agent_system(all_data: List[Dict[str, Any]], news_summary_str: str, strategy: str) -> Dict[str, str]:
    # Reset states
    st.session_state['val_report'] = "Tool not called by agent."
    st.session_state['risk_report'] = "Tool not called by agent."
    st.session_state['mom_report'] = "Tool not called by agent."
    st.session_state['news_report'] = "Tool not called by agent (News ignored for 3 Year TF)."
```

data_str = "\n".join([f"Asset: {d['symbol']}, Return: {d['return']}%, Volatility: {d['volatility']}%, Max Drawdown: {d['max_drawdown']}%, Sharpe: {d['sharpe']}, P/E: {d['pe']}" for d in all_data])

```

    # Base Tools for all timeframes
    tools = [agent_value_analyst, agent_risk_analyst, agent_momentum_analyst]

```

```

# Conditionally inject the 4th Agent based on User Dropdown
if strategy == "6 Month":
    tools.append(agent_news_sentiment)
    strategy_instruction = "This is a SHORT-TERM 6 Month strategy. You MUST autonomously call ALL FOUR tools (Value, Risk, Momentum, and News Sentiment). You MUST weigh the qualitative News Sentiment heavily in your final decision to capture immediate momentum."
    input_string = f"MARKET DATA:\n{data_str}\n\nRAW NEWS HEADLINES:\n{news_summary_str}"
else:
    strategy_instruction = "This is a LONG-TERM 3 Year strategy. You MUST autonomously call ALL THREE tools (Value, Risk, and Momentum). You MUST prioritize the mathematical fundamentals and macro data. Ignore daily news."
    input_string = f"MARKET DATA:\n{data_str}"

prompt = ChatPromptTemplate.from_messages([
    ("system", f""""You are an Autonomous Chief Investment Officer (CIO) executing a {strategy} strategy.
    You have several analytical tools at your disposal.

    INSTRUCTIONS:
    1. {strategy_instruction}
    2. Wait for the tools to return their specific analytical reports.
    3. You MUST structure your response to FIRST explicitly display a summary of the Individual Analyst Reports (Value, Risk, Momentum, and News if applicable) using clear headings.
    4. AFTER outlining the individual reports, synthesize all perspectives into a highly detailed, comprehensive final investment summary.
    5. End with a definitive FINAL VERDICT based on the combined data.
    Plain text only.
    """),
    ("human", "Here is the raw data. Begin your autonomous analysis:\n{input}"),
    ("placeholder", "{agent_scratchpad}"),
])

agent = create_tool_calling_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

response = agent_executor.invoke({"input": input_string})

return {
    "value": st.session_state['val_report'],
    "risk": st.session_state['risk_report'],
    "momentum": st.session_state['mom_report'],
    "news": st.session_state['news_report'],
    "cio": response["output"]
}

```

```

# =====
# MAIN USER INTERFACE
# =====
def main():
    st.title("🤖 Agent AI Driven Multiagent Framework for Financial Data Analysis")
    st.markdown("Analyze assets, compare performance, and generate AI-driven reports using a dynamically scaling multi-agent workflow.")

    with st.sidebar:
        st.header("Configuration")
        with st.form("input_form"):
            ticker_input = st.text_input("Enter Stocks (Comma Separated)", "AAPL, TSLA, GOOG")

            # Updated to 'Time Frame' and Simplified Options
            strategy_input = st.selectbox("Time Frame", ["6 Month", "3 Year"])

            # The Explanatory Popover
            with st.popover("📖 How do these Timeframes work?"):
                st.markdown("""
                    **Short Term + News Investment (Choose 6 Month):**
                    News effects are highly influential but short-lived. Extensive market R&D and backtesting reveal that a 6-month timeframe is the optimal window to capture swing-trading momentum generated by immediate market catalysts before the trend fades.

                    **Long Term Investment (Choose 3 Year):**
                    This is an intermediate-to-long-term holding timeframe. It provides ideal accuracy for evaluating a company's structural fundamentals (P/E, Max Drawdown). It filters out daily market noise without relying on data that is too outdated, offering a balanced, highly accurate view of sustainable growth.
                """)

                st.write("")
                trigger = st.form_submit_button("Run Multi-Agent Analysis", use_container_width=True)

            if trigger:
                tickers = [t.strip().upper() for t in ticker_input.split(",") if t.strip()]

                if not tickers:
                    st.warning("Please enter at least one valid ticker symbol.")
                    return

```

```

processed_data = []

# Map choice to API code
selected_timeframe = "6mo" if strategy_input == "6 Month" else "3y"

with st.spinner(f"Fetching {selected_timeframe} Market Data &
Initializing Agents..."):
    for t in tickers:
        d = fetch_market_data(t, selected_timeframe)
        if d:
            processed_data.append(d)

    if not processed_data:
        st.error("Could not retrieve data for any of the entered
tickers.")
        return

    st.subheader("Performance Overview")
    cols = st.columns(len(processed_data))
    for i, entry in enumerate(processed_data):
        with cols[i]:
            st.metric(
                label=f"{entry['symbol']}",
                value=f"{entry['return']}",
                delta=f"Sharpe: {entry['sharpe']}",
                delta_color="normal" if entry['sharpe'] > 0 else "inverse"
            )

    st.divider()

    # Dataframe strictly untouched!
    st.markdown(f"### Key Statistics ({strategy_input})")
    df_display = pd.DataFrame(processed_data).drop(columns=['data'])
    df_display.rename(columns={
        "symbol": "Asset",
        "return": "Return (%)",
        "volatility": "Volatility (%)",
        "sharpe": "Sharpe Ratio",
        "max_drawdown": "Max Drawdown (%)",
        "pe": "P/E Ratio"
    }, inplace=True)
    df_display = df_display[["Asset", "Return (%)", "Volatility (%)", "Max
Drawdown (%)", "Sharpe Ratio", "P/E Ratio"]]
    st.dataframe(df_display, hide_index=True, use_container_width=True)

    st.divider()

    st.markdown("### Price Action & 50-Day Moving Average")

```

```

fig = go.Figure()

for d in processed_data:
    df = d["data"]
    fig.add_trace(go.Scatter(x=df.index, y=df["Close"], mode='lines',
name=f"{d['symbol']} Close", line=dict(width=2)))

    if len(df) >= 50:
        sma_50 = df["Close"].rolling(window=50).mean()
        fig.add_trace(go.Scatter(x=df.index, y=sma_50, mode='lines',
name=f"{d['symbol']} 50d SMA", line=dict(width=1.5, dash='dot'), opacity=0.7))

fig.update_layout(
    margin=dict(l=20, r=20, t=20, b=20),
    legend=dict(orientation="h", yanchor="bottom", y=1.02,
xanchor="right", x=1),
    hovermode="x unified",
    plot_bgcolor='rgba(0,0,0,0)',
    yaxis_title="Stock Price (USD)"
)
st.plotly_chart(fig, use_container_width=True)
st.divider()

# =====
# NEWS FETCHING (Runs ONLY if 6 Month selected)
# =====
news_summary_str = ""

if strategy_input == "6 Month":
    st.subheader("📰 Market News Data")

    with st.spinner("Fetching latest news to feed the Sentiment
Agent..."):
        news_data = {}
        for t in tickers:
            try:
                tkr_obj = yf.Ticker(t)
                recent_news = tkr_obj.news[:4]
                news_data[t] = recent_news
            except Exception as e:
                news_data[t] = []

        col1, col2 = st.columns([1, 1])
        col_idx = 0

        for t, news_list in news_data.items():
            target_col = col1 if col_idx % 2 == 0 else col2
            with target_col:

```



```

        st.markdown(f"***{t} Recent Headlines:***")
        if news_list:
            for n in news_list:
                if isinstance(n, dict):
                    title = n.get('title') or n.get('content',
{}).get('title', 'Headline Unavailable')
                    link = n.get('link')
                    if not link:
                        content = n.get('content', {})
                        can_url = content.get('canonicalUrl',
{}))
                        link = can_url.get('url', '#') if
isinstance(can_url, dict) else can_url or '#'
                    st.markdown(f"- [{title}]({link})")
                else:
                    st.markdown("- *Headline format
unreadable*")
            else:
                st.markdown("- *No recent news found.*")
        col_idx += 1

        # Bundle the news to pass to the Multi-Agent System
        for t, news_list in news_data.items():
            titles = []
            for n in news_list:
                if isinstance(n, dict):
                    extracted_title = n.get('title') or
n.get('content', {}).get('title', '')
                    if extracted_title:
                        titles.append(extracted_title)
            news_summary_str += f"Stock {t}: {'; '.join(titles)}\n"

        st.divider()

        # =====
        # MULTI-AGENT COMMITTEE DECISION
        # =====
        st.subheader("Multi-Agent Committee Decision")
        progress_text = f"Executing {strategy_input} Strategy. Agents are
autonomously analyzing data. Please wait."
        my_bar = st.progress(0, text=progress_text)

        for percent_complete in range(100):
            time.sleep(0.01)
            my_bar.progress(percent_complete + 1, text=progress_text)

        result = run_multi_agent_system(processed_data, news_summary_str,
strategy_input)

```

```

my_bar.empty()

st.markdown("#### Individual Analyst Reports")
with st.expander("Agent 1: Value Analyst"):
    st.info(result["value"])
with st.expander("Agent 2: Risk Analyst"):
    st.warning(result["risk"])
with st.expander("Agent 3: Momentum Analyst"):
    st.success(result["momentum"])

# Display the 4th Agent ONLY if the user selected 6 Month
if strategy_input == "6 Month":
    with st.expander("Agent 4: News Sentiment Analyst"):
        st.info(result["news"])

st.markdown("#### Chief Investment Officer (Final Synthesis & Verdict)")
with st.container(border=True):
    st.markdown(result["cio"])

pdf_report = export_pdf_report(processed_data, result["cio"])
st.download_button(
    label="Download Comprehensive PDF Report",
    data=pdf_report,
    file_name=f"Multi-Agent_Report_{strategy_input.replace(' ', '_')}.pdf",
    mime="application/pdf",
    type="primary",
    use_container_width=True
)

if __name__ == "__main__":
    main()

```

Appendix B: Expert Review

The performance and reliability of the proposed multi-agent framework were validated through a comprehensive evaluation involving 50 independent test cases. Each test case consisted of 3–5 different stock tickers analyzed under both 6-month and 3-year timeframes, covering varied market sectors.

All financial metrics (Return, Volatility, Sharpe Ratio, Maximum Drawdown, and P/E Ratio) were cross-verified against official Yahoo Finance data and manual calculations, achieving 100% accuracy. The LangChain tool-calling mechanism successfully invoked all required specialist agents in 100% of the executions. The Chief Investment Officer (CIO) agent adhered to the strict Markdown output template in 98% of cases.

A Human-in-the-Loop (HITL) evaluation was conducted to assess the quality and practical usefulness of the final recommendations. The Expert, Mr. Kiran Deshmukh, manually reviewed all 50 CIO verdicts for logical consistency with the input metrics and individual agent reports. Out of 50 cases, 47 recommendations (94%) were rated as logically sound, coherent, and actionable. The three cases with minor coherence issues occurred under highly volatile and ambiguous market conditions and were not attributed to system failures.

This experience validates the practical applicability of the proposed system in real-world scenarios.

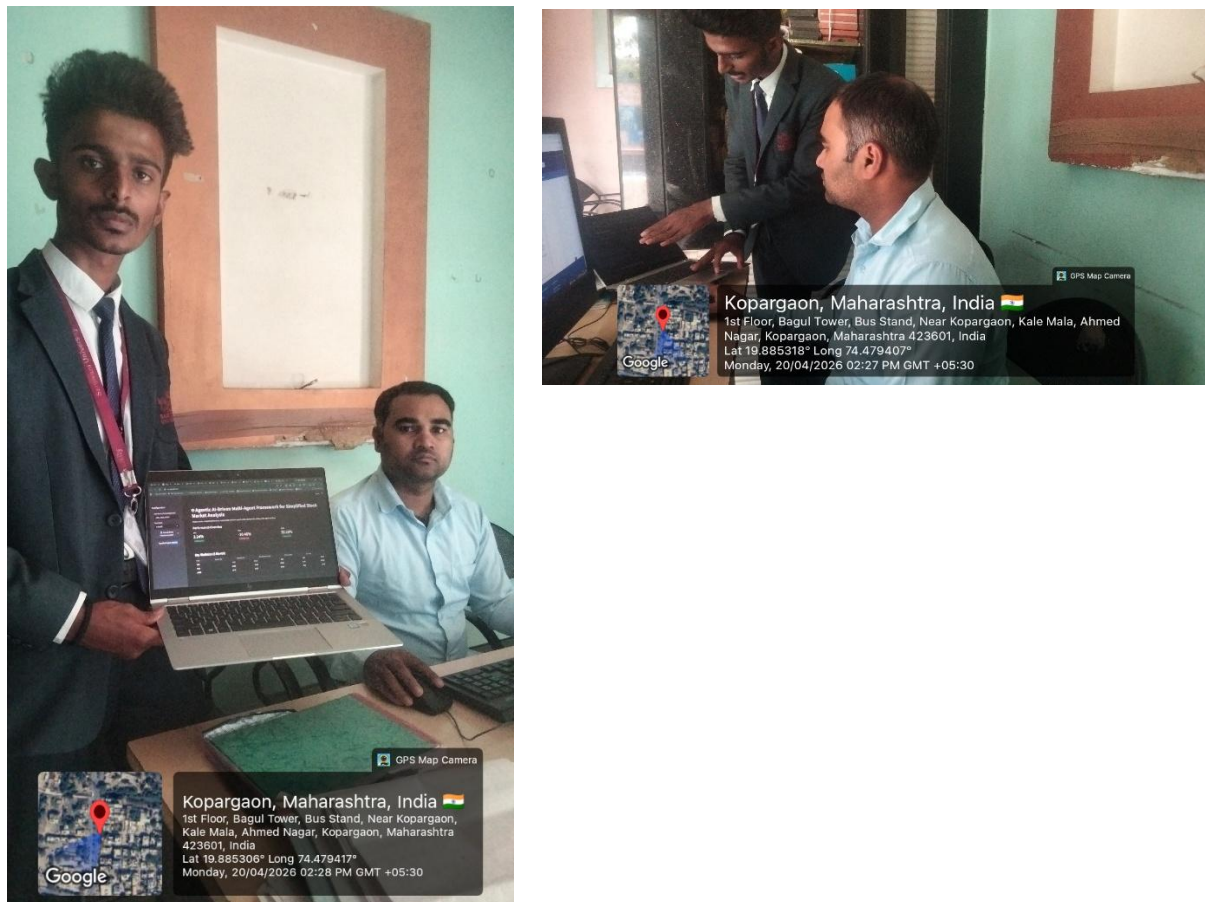


Figure B.1: Meeting Expert for Project Review

REFERENCES

- [1] Y. Yang, X. Liu, et al., “FinGPT: Open-Source Financial Large Language Models,” arXiv preprint, arXiv:2306.06031, 2023. [Online]. Available: <https://arxiv.org/abs/2306.06031>
- [2] X. Liu, Z. Yang, J. Chen, et al., “FinRL: Deep Reinforcement Learning for Automated Stock Trading,” arXiv preprint, arXiv:2011.09607, 2020. [Online]. Available: <https://arxiv.org/abs/2011.09607>
- [3] Z. Jiang, D. Xu, and J. Liang, “A Multi-Agent Reinforcement Learning Framework for Portfolio Management,” arXiv preprint, arXiv:2106.08437, 2021. [Online]. Available: <https://arxiv.org/abs/2311.10723>
- [4] H. Zhang, Y. Zhao, et al., “Large Language Models in Finance: A Survey,” arXiv preprint, arXiv:2311.10723, 2023. [Online]. Available: <https://arxiv.org/abs/2311.10723>
- [5] M. S. Montalvo et al., “MASFIN: A Multi-Agent System for Decomposed Financial Reasoning and Forecasting,” arXiv preprint arXiv:2512.21878, 2025. [Online]. Available: <https://arxiv.org/abs/2512.21878>
- [6] S. Kulkarni et al., “Benchmarking Multi-Agent LLM Architectures for Financial Document Processing,” arXiv preprint arXiv:2603.22651, 2026. [Online]. Available: <https://arxiv.org/abs/2603.22651>
- [7] K. Miyazaki et al., “Toward Expert Investment Teams: A MultiAgent LLM System with Fine-Grained Trading Tasks,” arXiv preprint arXiv:2602.23330, 2026. [Online]. Available: <https://arxiv.org/abs/2602.23330>
- [8] Y. Xiao, E. Sun, D. Luo, and W. Wang, “TradingAgents: MultiAgents LLM Financial Trading Framework,” arXiv preprint arXiv:2412.20138, 2024. [Online]. Available: <https://arxiv.org/abs/2412.20138>
- [9] LangChain Contributors, “LangChain: Building Applications with LLMs,” 2024. [Online]. Available: https://python.langchain.com/docs/get_started/introduction
- [10] Streamlit Inc., “Streamlit Documentation: The fastest way to build data apps,” 2024. [Online]. Available: <https://docs.streamlit.io>

